

Numerical Code Recognition on an Industrial Seal

BTHA 2026 Summer School

September 7–11, 2026

Marek Hrúz, Petr Neduchal

Contents

1	Introduction	3
2	Solution Using Classical Image Processing	4
2.1	Introduction to the Dataset	4
2.2	Image Preprocessing	5
2.2.1	Noise Reduction	5
2.2.2	Edge Detection (optional)	5
2.2.3	Image Thresholding	6
2.2.4	Component Detection	6
2.3	Digits recognition and classification	6
2.3.1	Recommended Preprocessing	7
2.3.2	Good Feature Groups for Classical Numeral Recognition	7
2.3.3	Classification	11
3	Solution Using a Convolution Neural Network	12
3.1	Dataset Preparation	12
3.2	Dataset and Data Loading	13
3.3	Data Augmentation	13
3.4	CNN Architecture	13
3.5	Training the Network	14
3.6	Training and Validation Analysis	14
3.7	Final Evaluation	14
4	Solution Using a Multimodal Large Language Model	16
4.1	Preparing the Image	16
4.2	Calling a Vision-Language Model	17
4.3	Prompt Design	17
4.4	Output Format	17
4.5	Zero-Shot and Few-Shot Prompting	17
4.6	Repeated Recognition and Voting	18
4.7	Plausibility and Post-processing	18
5	Comparison of Recognition Approaches	19
6	Final Task: Recognition Competition	20
6.1	Instructions for the Final Competition	20
6.1.1	Required Files	21
6.1.2	main.py	21

6.1.3	Dependencies	22
6.1.4	Dataset and Helper Code	22
6.1.5	Final Submission	22

Chapter 1

Introduction

This task focuses on the recognition of numerical codes printed on industrial security seals. You will work with a dataset consisting of seal images together with labels and a separate dataset containing individual digit images. The objective is to develop a system capable of accurately reading the numerical code from a seal image.

The task is divided into three independent parts, each demonstrating a different approach to solving the same computer vision problem. The first part explores a traditional image processing pipeline without the use of neural networks, relying on techniques such as image enhancement, thresholding, segmentation, and classical machine learning. The second part introduces a lightweight convolutional neural network implemented in PyTorch and demonstrates how modern deep learning methods can simplify and improve the recognition task. The third part investigates the use of multimodal large language models (LLMs), showing how foundation models can perform visual recognition with little or no task-specific training through appropriate prompt engineering.

In the final exercise, you can combine the acquired knowledge into a complete application capable of recognizing seal codes from images or a live camera stream.

Chapter 2

Solution Using Classical Image Processing

In this section, you will solve the recognition problem using only traditional image processing and classical machine learning techniques. The objective is to understand how far a carefully designed image processing pipeline can go without relying on neural networks. Starting from the original seal image, you will gradually build a complete solution consisting of image preprocessing, thresholding, digit segmentation, feature extraction, and digit classification.

2.1 Introduction to the Dataset

The first task is to load an image from the provided dataset and display it. You should become familiar with basic image handling in Python and verify that the selected image has been loaded correctly before proceeding with further processing.

For image loading, participants may use the `cv2.imread`¹ function from the OpenCV library. Since OpenCV stores color images in the BGR color space, while Matplotlib package expects the RGB format, the displayed image may appear with incorrect colors if no conversion is performed. Therefore, you should convert the image from BGR to RGB before visualization using an appropriate OpenCV function, such as `cv2.cvtColor`.

The image should then be displayed using Matplotlib `pyplot`². You should create a simple helper function that visualizes an image, sets an appropriate figure size and title, and hides the axis labels. Such a function will simplify the visualization of intermediate results throughout the following tasks.

```
im = cv2.imread(path_to_image)
% image processing
show_image("Seal image", im) # your visualization
```

Example 2.1: Image load and show example

¹<https://opencv.org/read-display-and-write-an-image-using-opencv/>

²https://matplotlib.org/3.5.3/api/_as_gen/matplotlib.pyplot.html

In the next step you should verify the image shape (height, width, and number of color channels), identify the underlying data type, and determine the minimum and maximum pixel values. These values can be obtained directly from the **NumPy** array or by using standard **NumPy** functions³.

The next step is to investigate the individual color channels. You should separate the image into its blue, green, and red channels using an appropriate **OpenCV** function (e.g., `cv2.split`). Each channel should then be visualized independently as a grayscale image using **Matplotlib**. By comparing the three channels, participants can observe how each color component contributes to the final color image and gain a better understanding of the internal image representation used in computer vision.

Instead of working with the individual color channels, you can also convert the input image directly to a grayscale representation using `cv2.cvtColor` function with the appropriate color conversion flag.

Before proceeding with the processing tasks, you should familiarize yourself with the dataset. Feel free to load the provided labels from image names, visualize several randomly selected image samples, and inspect the overall quality of the data. Particular attention should be paid to variations in illumination, image quality, and other factors that may influence the performance of subsequent image processing algorithms.

2.2 Image Preprocessing

It is often a good idea to apply several preprocessing operations before performing more advanced image analysis. These techniques help reduce unwanted noise, enhance important image features, and improve the robustness of subsequent computer vision algorithms. In this section, you will explore several commonly used preprocessing methods and observe how they influence the appearance of the image.

2.2.1 Noise Reduction

You can reduce image noise to suppress small intensity variations that may negatively affect the results of subsequent operations. A common approach is to apply a Gaussian blur, which smooths the image while preserving its overall structure. Gaussian filtering is frequently used as a preprocessing step before thresholding, edge detection, or contour extraction. You can use the `cv2.GaussianBlur`⁴ function to experiment with different kernel sizes and observe how the amount of smoothing influences the image.

2.2.2 Edge Detection (optional)

After preprocessing the image, you should detect object boundaries using an edge detection algorithm. One of the most widely used methods is the Canny edge detector, which identifies locations with strong intensity changes while suppressing noise-induced responses. The quality of the detected edges depends on the amount of image smoothing and the selected threshold values, so you are encouraged to experiment with these

³<https://numpy.org/doc/stable/reference/index.html>

⁴https://docs.opencv.org/4.13.0/d4/d13/tutorial_py_filtering.html

parameters and observe their influence on the resulting edge map. The implementation is available through the `cv2.Canny`^{5,6} function.

2.2.3 Image Thresholding

Thresholding converts a grayscale image into a binary image by assigning each pixel to either the foreground or the background based on an intensity threshold. You should apply Otsu's thresholding, which automatically estimates an appropriate threshold value from the image histogram without requiring manual tuning. This operation is useful for separating objects of interest from the background and serves as the basis for many segmentation tasks. The `cv2.threshold`⁷ function with the `THRESH_OTSU` flag can be used to perform this operation.

2.2.4 Component Detection

Once a binary mask has been created, you can identify individual foreground regions using connected-component analysis. Before detecting the components, it is useful to clean the mask using basic morphological operations. You can create a suitable structuring element with `cv2.getStructuringElement`⁸ and then apply opening and closing operations with `cv2.morphologyEx`⁹ to remove small isolated regions and fill small gaps inside objects.

The individual regions can then be detected using connected component function^{10,11}. This function assigns a unique label to each connected component and provides useful properties such as its area, bounding box, and centroid. You should also remove very small components using a minimum-area threshold, since these regions often correspond to noise rather than relevant objects. The remaining components can then be stored and optionally sorted according to their size for further processing. Maybe good idea is to search for six components that lie in a row. As a next step, you may try to identify a group of six components that are approximately aligned in a row.

2.3 Digits recognition and classification

After detecting the individual numeral components, the next step is to recognize and classify them. Unlike the previous image-processing steps, there is no single recommended solution for this task. You can choose from several classical approaches and experiment with different combinations of preprocessing, feature extraction, and classification methods.

The following sections provide a set of possible techniques that can be used to describe the shape and structure of individual numerals. You do not need to implement all of them. Instead, select one or more approaches, evaluate how well they represent

⁵<https://opencv.org/edge-detection-using-opencv/>

⁶https://docs.opencv.org/4.13.0/da/d22/tutorial_py_canny.html

⁷<https://opencv.org/image-thresholding-using-opencv/>

⁸https://docs.opencv.org/4.13.0/d4/d76/tutorial_js_morphological_ops.html

⁹<https://www.geeksforgeeks.org/python/python-opencv-morphological-operations/>

¹⁰`cv2.connectedComponentsWithStats`

¹¹<https://pyimagesearch.com/2021/02/22/opencv-connected-component-labeling-and-analysis/>

the numerals in the provided dataset, and consider combining multiple feature types if a single descriptor is not sufficient.

2.3.1 Recommended Preprocessing

Before extracting features, it is recommended to normalize every detected numeral so that the resulting feature representation is as consistent as possible:

- Convert to binary foreground/background.
- Crop tightly to the numeral bounding box.
- Preserve aspect ratio and center it on a fixed canvas, for example 32×32 .
- Optionally deskew using image moments.
- Ensure consistent polarity: numeral pixels = 1, background = 0.

Normalization is often more important than the choice of classifier.

2.3.2 Good Feature Groups for Classical Numeral Recognition

To classify numerals using classical machine-learning methods, you first need to transform each normalized numeral image into a set of descriptive numerical features. Different feature extraction methods capture different properties of the numeral, such as its overall shape, spatial distribution, topology, or edge structure. The following approaches provide several possible feature groups that you can explore individually or combine into a single feature vector.

1. Projection Profiles

Compute the number of foreground pixels in each row and each column.

Horizontal projection:

$$h_i = \sum_x I(x, i)$$

Vertical projection:

$$v_j = \sum_y I(j, y)$$

Typical features:

- Horizontal projection profile
- Vertical projection profile
- Downsampled projections (e.g., 8 horizontal + 8 vertical bins)

Advantages:

- Very simple to implement.
- Captures where the numeral is wide or narrow.

- Works well for fixed fonts.

2. Zoning (Grid Occupancy)

Divide the normalized image into a regular grid (e.g., (4 4)) and compute the foreground density inside each cell.

$$f_{r,c} = \frac{1}{|Z_{r,c}|} \sum_{(x,y) \in Z_{r,c}} I(x,y)$$

Typical grids:

- $(4 \times 4) \rightarrow 16$ features
- $(6 \times 6) \rightarrow 36$ features

Advantages:

- Preserves coarse spatial information.
- Easy to visualize.
- Robust to small deformations.

3. Image Moments

Compute normalized central moments or Hu invariant moments.

Central moments:

$$\mu_{pq} = \sum_x \sum_y (x - \bar{x})^p (y - \bar{y})^q I(x, y)$$

Possible features:

- Centroid
- Variance in x
- Variance in y
- Covariance
- Orientation
- Eccentricity
- Hu moments

Advantages:

- Compact global shape descriptor.
- Translation and scale invariant.
- Widely used classical feature.

4. Contour Geometry

Extract the outer contour and compute geometric descriptors.

Typical features:

- Area
- Perimeter
- Bounding-box width
- Bounding-box height
- Aspect ratio
- Extent
- Circularity

$$C = \frac{4\pi A}{P^2}$$

- Convex hull area
- Solidity

$$\text{Solidity} = \frac{A}{A_{\text{convex hull}}}$$

Advantages:

- Describes the overall shape.
- Easy to explain.
- Useful for distinguishing thin and compact numerals.

5. Topological Features

Describe the topology of the numeral.

Typical features:

- Number of connected components
- Number of holes
- Hole area
- Hole centroid
- Relative hole position

Examples:

- 0 → one hole
- 6 → one lower hole
- 8 → two holes
- 9 → one upper hole

Advantages:

- Extremely discriminative.
- Highly interpretable.
- Nearly invariant to small deformations.

6. Scan-Line Intersection Features

Place several horizontal and vertical scan lines across the image.

For each scan line compute:

- Number of foreground segments
- Number of transitions
- Length of foreground intervals
- Position of first and last intersection

Advantages:

- Captures structural information.
- Simple implementation.
- Easy for students to understand.

7. Symmetry Features

Measure similarity between mirrored halves of the numeral.

Possible features:

- Vertical symmetry
- Horizontal symmetry
- Upper/lower symmetry
- Left/right symmetry

Example metric:

$$S = \frac{1}{N} \sum |I(x, y) - I(W - 1 - x, y)|$$

Advantages:

- Helps distinguish symmetric and asymmetric digits.
- Very intuitive feature.

8. Distance Transform Features

Compute the Euclidean distance transform inside the foreground.

Possible features:

- Maximum distance
- Mean distance
- Variance
- Histogram of distances
- Regional averages

Advantages:

- Encodes stroke thickness.
- Robust to small contour variations.

9. Histogram of Oriented Gradients (HOG)

Compute histograms of local edge orientations over small cells.

Typical parameters:

- 9 orientation bins
- 8×8 pixel cells
- 2×2 cell blocks

Advantages:

- Excellent recognition performance.
- Robust to small shifts.
- Classical handcrafted feature descriptor.
- Often paired with a linear SVM.

2.3.3 Classification

Once you have extracted a suitable feature vector for each numeral, you can use these features to perform the final classification. You are free to select an appropriate classical machine-learning method, such as **k-Nearest Neighbors (k-NN)**, a **Support Vector Machine (SVM)**, or another classifier of your choice^{12,13}.

¹²<https://www.kaggle.com/code/faressayah/practical-guide-to-6-classification-algorithms>

¹³https://scikit-learn.org/stable/supervised_learning.html

Chapter 3

Solution Using a Convolution Neural Network

In this section, you will solve the same recognition task using a custom convolutional neural network^{1,2,3}. Instead of manually designing feature extraction and classification algorithms, the model will learn discriminative features directly from annotated training data. You will prepare the dataset, implement a lightweight neural network in PyTorch, train and evaluate the model.

The complete solution consists of several steps. You will first prepare the dataset and divide it into appropriate subsets, then create a data-loading pipeline, design a CNN architecture, train the network, and finally evaluate its recognition performance on previously unseen images.

3.1 Dataset Preparation

Before training a neural network, the available numeral images should be divided into separate **training, validation, and test sets**. The training set is used to optimize the network parameters, the validation set can be used during development to evaluate the model and tune its configuration, while the test set should remain unseen until the final evaluation.

A possible split is 80% for training, 10% for validation, and 10% for testing. Since this is a multi-class classification problem with ten numeral classes, you should perform the split separately for each class. This ensures that all numerals remain appropriately represented in each subset. It is also a good idea to use a fixed random seed so that the same split can be reproduced later.

You may store the resulting subsets as simple manifest files containing the image path and corresponding class label. Before continuing, inspect the number of samples belonging to each class and verify that the dataset is reasonably balanced.

¹<https://www.geeksforgeeks.org/deep-learning/convolutional-neural-network-cnn-in-machine-learning/>

²<https://www.geeksforgeeks.org/deep-learning/building-a-convolutional-neural-network-using-pytorch/>

³https://docs.pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html

3.2 Dataset and Data Loading

The next step is to create a data-loading pipeline suitable for neural-network training. In **PyTorch**, you can implement a custom dataset class by deriving from `Dataset` class⁴ and implementing the `__len__` and `__getitem__` methods.

Each sample should provide an image together with its corresponding class label. Since the task is based primarily on the shape of the numeral, the images can be loaded as **single-channel grayscale images**. They should also be converted to tensors and their pixel values represented in a suitable numerical range.

Neural networks process batches of images and therefore require all input samples to have the same dimensions. Resize the numeral images to a fixed resolution, for example 28×28 pixels. However, be careful not to unnecessarily distort the shape of a numeral. One possibility is to preserve its original aspect ratio, add padding where necessary, and then resize the resulting image to the required dimensions.

Finally, use `torch.utils.data.DataLoader` to organize the samples into batches. The training dataset should normally be shuffled between epochs, while validation and test datasets should retain a deterministic order.

3.3 Data Augmentation

The amount and variability of training data can have a significant influence on the ability of a neural network to generalize to previously unseen images. You may therefore experiment with **data augmentation**, where slightly modified versions of the original training samples are generated during training.

Consider transformations that represent realistic variations in the dataset, such as small translations, rotations, changes in scale, or moderate variations in image intensity. Avoid transformations that substantially change the shape or meaning of the numeral.

Augmentation should normally be applied only to the **training set**. Validation and test images should undergo only the preprocessing necessary to produce the expected network input.

3.4 CNN Architecture

Design a small convolutional neural network that accepts a grayscale numeral image and predicts one of the ten classes (0–9). You do not need a large or complicated architecture for this task.

A reasonable starting point is to use several `Conv2d`⁵ layers for extracting visual features. Each convolution can be followed by a nonlinear activation function such as `ReLU`, while `MaxPool2d` layers can gradually reduce the spatial resolution of the feature maps. After the convolutional part, flatten the extracted features and process them using one or more fully connected layers.

The final layer should produce ten output values, one for each numeral class. You may also experiment with regularization techniques such as `Dropout` to reduce overfitting.

⁴`torch.utils.data.Dataset`

⁵<https://docs.pytorch.org/docs/2.13/nn.html>

Start with a simple architecture and modify the number of convolutional layers, channels, kernel sizes, or fully connected layers if necessary. A more complicated network does not automatically provide better results, particularly when the available dataset is relatively small.

3.5 Training the Network

Training consists of repeatedly presenting batches of images to the network, comparing its predictions with the correct labels, and updating its parameters.

For every training batch, perform a forward pass through the network and calculate the classification loss. For a multi-class problem such as numeral recognition, `CrossEntropyLoss` is a suitable choice. Then perform backpropagation using `loss.backward()` and update the network parameters using an optimizer such as `Adam`.

Repeat this process for several epochs. During training, monitor at least the average loss and classification accuracy. If a validation set is available, evaluate the model on it after each epoch without updating the network parameters.

If a CUDA-compatible GPU is available, you can move both the model and input tensors to the GPU using PyTorch. However, the proposed network is intentionally small enough that it can also be trained on a CPU.

3.6 Training and Validation Analysis

Do not evaluate the network only according to its final training accuracy. Compare the evolution of training and validation loss and accuracy over the individual epochs.

If training accuracy continues to improve while validation performance becomes worse, the model may be overfitting the training data. You can then experiment with data augmentation, dropout, a smaller network, or other regularization techniques.

Conversely, poor performance on both the training and validation sets may indicate that the network is too simple, the training duration is insufficient, or the input preprocessing needs improvement.

Try changing selected hyperparameters such as the **learning rate**, **batch size**, **number of epochs**, **input image resolution**, or **network architecture**, and observe how these changes influence the results.

3.7 Final Evaluation

Once you have selected the final model configuration, evaluate it on the test set. The test data should not have been used for network training or for selecting hyperparameters, so its performance provides a better estimate of how well the model generalizes to unseen numeral images.

In addition to the overall classification accuracy, you may investigate which numeral classes are most frequently confused. A **confusion matrix** can help identify systematic classification errors and reveal whether particular numeral pairs are difficult for the network to distinguish.

Finally, compare the CNN-based solution with the classical recognition approach from the previous task. Consider not only the achieved accuracy, but also the amount of preprocessing required, implementation complexity, interpretability, and robustness to variations in the input images.

Chapter 4

Solution Using a Multimodal Large Language Model

In the previous tasks, you designed the complete recognition pipeline yourself, either by extracting handcrafted features or by training a convolutional neural network. A different approach is to use a pretrained **vision-language model (VLM)** that can directly analyze the input image and return the numerical code without explicitly implementing digit segmentation, feature extraction, or classifier training.

In this task, you will explore how a general-purpose multimodal model can be used as an OCR system for industrial seal images. The main challenge is no longer the design and training of the recognition model itself, but rather the preparation of the input, the design of an effective prompt, and the interpretation and validation of the model output.

4.1 Preparing the Image

The vision-language model needs to receive the seal image together with the textual instructions. Depending on the selected API, the image can be provided directly as an image file or encoded into a suitable representation, such as a Base64-encoded image¹.

You may use OpenCV to load and optionally preprocess the image before sending it to the model. For example, `cv2.imread` can be used to load the image and `cv2.imencode` to encode the processed image into PNG or JPEG format. The resulting data can then be converted to Base64² if required by the selected API.

At first, try using the original image without additional preprocessing. Later, you can investigate whether cropping the region containing the seal number, improving contrast, converting the image to grayscale, or applying other preprocessing techniques from the previous tasks improves the recognition result.

¹Suitable for Ollama

²HINT: `base64.b64encode(buffer).decode('utf-8')`

4.2 Calling a Vision-Language Model

Use a vision-capable language model that accepts both **text and image input**³. The image provides the visual information, while the accompanying text describes what the model should extract from it.

The provided implementation demonstrates two possible ways of accessing such a model: through an external API or through an Ollama-based server. The exact interface is not important for the recognition task itself. In both cases, the basic principle remains the same: provide the image together with instructions and obtain the model response.

When experimenting with different models, keep the input image and prompt unchanged whenever possible. This allows you to compare their recognition performance under similar conditions.

4.3 Prompt Design

The quality of the prompt can significantly influence the recognition result. Instead of simply asking the model to "read the image", clearly describe the task and specify what information should be extracted.

Experiment with different prompt formulations and observe how small changes in the instructions influence the output.

4.4 Output Format

For automatic processing, the model output should be predictable and easy to parse. One possibility is to instruct the model to return only the detected numerical code. A more robust approach is to request a **structured response** containing predefined fields.

For example, the response may contain the detected seal number together with an optional note or information about the recognition result. When supported by the selected API, you can define the expected response structure using a schema or a data model and require the LLM to follow this format.

Structured output makes it easier to integrate the LLM into a larger processing pipeline and prevents additional explanatory text from interfering with automatic evaluation.

4.5 Zero-Shot and Few-Shot Prompting

Start with a **zero-shot** approach, where the model receives only the task description and the image to be recognized. This provides a useful baseline and shows how well the pretrained model understands the task without additional examples.

You may then experiment with **few-shot prompting** by providing examples of correctly recognized seal numbers and their expected outputs. Examples can help the model understand the expected format and the visual characteristics of the task.

³Example: `gemma3:12b`

Compare whether additional examples actually improve recognition. More complex prompts do not necessarily produce better results, so the effectiveness of each modification should be evaluated experimentally.

4.6 Repeated Recognition and Voting

LLM outputs may not always be perfectly consistent. One possible strategy is to perform recognition multiple times, possibly using different prompts, image variants, or models, and compare the resulting predictions.

If several predictions agree on the same numerical code, their agreement can be used as additional evidence for the result. You may implement a simple **majority-voting strategy** or compare predictions digit by digit. Disagreement between predictions can also be used to identify potentially uncertain samples.

4.7 Plausibility and Post-processing

The raw LLM prediction does not necessarily have to be accepted directly. If you know something about the expected seal number, you can apply additional rules to validate the result.

For example, you may check whether the output contains the expected number of digits, whether all characters are valid numerical symbols, or whether the predicted value follows a known format. Predictions that violate these constraints can be rejected or marked as uncertain.

Think about which constraints can be safely applied without accidentally correcting a valid but unexpected seal number.

Chapter 5

Comparison of Recognition Approaches

After completing all three recognition approaches, compare their performance on the same set of test images. Focus primarily on **recognition accuracy** and **processing time**, and discuss the advantages and limitations of each solution. Consider whether the additional complexity of a CNN or the computational requirements of a vision-language model are justified by their recognition performance compared with the classical image-processing approach. Based on your results, determine which method would be the most suitable for a practical seal recognition system.

Chapter 6

Final Task: Recognition Competition

As the final task, you will take your selected recognition solution and prepare it to run on a laptop provided by the organizers. Your solution must be able to run on this computer without requiring any manual intervention during the evaluation.

The organizers will provide a test dataset containing previously unseen seal images. The dataset will be placed in a predefined directory on the evaluation laptop. Your solution must process all images from this directory and generate a CSV file containing at least the filename of each input image and the numerical code detected on the seal.

The organizers will use the generated CSV file to automatically evaluate the recognition accuracy of your solution against the ground-truth annotations. The processing time of your solution will also be measured during the evaluation.

Your team will be evaluated according to three criteria:

- Speed – how quickly your solution processes the complete test dataset.
- Accuracy – how many seal numbers are recognized correctly.
- Number of implemented approaches – how many of the three recognition approaches described in this document you successfully implement and make available for the competition (1–3).

The individual criteria will be converted into a final score and combined into the overall competition result. The team with the highest total score wins the competition.

The goal is therefore not simply to achieve the highest recognition accuracy, but to find a good balance between accuracy, processing speed, and the number of implemented recognition approaches.

6.1 Instructions for the Final Competition

For the final competition, you should select **one of the recognition approaches described above**, or combine multiple approaches if you have implemented them. Your solution must be prepared so that it can be executed automatically on the evaluation computer provided by the organizers.

6.1.1 Required Files

Your submission must contain at least the following files:

- `main.py` – the main script used to run your recognition solution.
- `requirements.txt` – a list of all additional Python dependencies required by your solution.
- Information about the **Python version** required to run your solution.

Please specify which Python version your solution requires, for example:

Python 3.12

The organizers will use this information to prepare the appropriate Python environment on the evaluation computer.

The basic environment already contains the following libraries:

- PyTorch (`torch`)
- TorchAudio (`torchaudio`)
- TorchVision (`torchvision`)
- OpenCV (`opencv`)
- NumPy (`numpy`)

Therefore, you **do not need to include these packages in `requirements.txt`**. Include only additional packages that your solution requires.

The evaluation computer provides **CUDA 12.8**.

6.1.2 `main.py`

The `main.py` script must accept two command-line parameters:

```
--input-dir  
--output-dir
```

`--input-dir` specifies the path to the directory containing the input dataset. The directory will contain the PNG images that your solution must process.

`--output-dir` specifies the path to the directory where your solution should store its results.

For example:

```
python main.py --input-dir /path/to/input --output-dir /path/to/output
```

Your script must process **all PNG images** in the input directory and generate a CSV file in the specified output directory.

The CSV file must be named according to your team name. For example, if your team is called `team1`, the output file must be:

`team1.csv`

The CSV file must contain at least:

- the filename of the input image,
- the numerical code recognized by your solution.

The exact same CSV format is provided in the competition dataset.

6.1.3 Dependencies

If your solution requires additional Python packages beyond the basic environment, list all of them in `requirements.txt`.

For example, if your solution uses `scikit-image` or `scikit-learn`, these packages should be included in the requirements file.

Your solution should not depend on packages that are not listed in `requirements.txt` or already available in the basic environment.

6.1.4 Dataset and Helper Code

The dataset used during the summer school is available at:

Dataset:

<https://drive.google.com/drive/folders/1Qqe-v9kx0x0U4B-9VyEzCtE46gWeP7v1?usp=sharing>

Additional helper code and examples are available in the following repository:

Helper code repository:

https://github.com/neduchal/AI_Summer_School_2026

You can use the provided helper code as a starting point for your implementation, but your final competition solution must be contained in your `main.py` and be executable using the specified interface.

6.1.5 Final Submission

Before submitting your solution, make sure that:

1. `main.py` is present and can be executed from the command line.
2. `main.py` accepts both `-input-dir` and `-output-dir`.
3. All PNG images in the input directory are processed.
4. The required CSV file is generated in the output directory.
5. The CSV file is named according to your team name.
6. All additional dependencies are listed in `requirements.txt`.

7. The required Python version is clearly specified.
8. The solution does not require manual interaction during execution.
9. The solution works in the provided evaluation environment.
10. The solution can use GPU acceleration if appropriate.
11. The solution is ready to be copied to and executed on the organizers' laptop - using flash drive.